

Task #1: Write a program that takes a filename *f* as input, and checks whether file *f* contains at least 2 lines each of which has one word with the following pattern: the word starts with a capital letter (i.e., A-Z) and ends with a small letter vowel (i.e., a,e,i,o, or u). As an example, "Apple", "Pen2Pe", or so can be such a word. Your program needs to print whether the file satisfies the required pattern. Also, it prints each line (in file *f*) that contains a word conforming the above pattern. Your program should also print the line numbers of the matching lines. *Writing readme carries 1 point.*

Example run: ruby task1.rb file-to-search-in An example output is as follows.

Line 3: sdf **Apple** bjAbc bjb

Line 7: Bgjh 12bask 13hj **Pen2Pe**

Line 10: abcd **Xo** sdf76as

This file has 3 lines which satisfy the required pattern.

Note: 1. Your project repository needs to include a sample file-to-search-in. 2. Do NOT use any library's grep function. You implement it on your own, which may take only few lines of code. You can utilize *regular expression*.

Task #2: Write a function bar(*n*) to compute the value of y^y without using the exponent operator, (i.e., your need to use the loop syntax) whereas *y* is a positive integer. Test your program at least for 2 values of *y* such as 5, and 6.

Task #3: Write a function that takes an array *A* of integers as the input and does the following: (a) Uses *each* method to print the least significant digit of each integer in *A*, (b) Uses *select* method to find all the integers (in *A*) which are less than 4, (c) Uses *map* method to build a new array with the cubes of the integers of *A*, and (d) Uses *inject* method to find the sum of the squares of the integers of *A*. To test the function, build an array *A* of 15 random integers, and pass *A* into the function as a parameter. *Writing readme carries 1 point.*

Task #4: Function4A calculates the Fibonacci Series ($F_n = F_{n-1} + F_{n-2}$) iteratively. Calculate all Fibonacci numbers (F_n) for $n = 1$ to 40. Function4B calculates the same Fibonacci Series but using the recursion technique. After implementing the above two functions, compare the computation time (for doing the above calculation) using Ruby's benchmark library. *Writing readme carries 1 point.* **Hint:** if necessary, Ruby lecture slides (ppt) can help you write the iterative version.

Task#5: The Tree class presented in the textbook (Day2) chapter is interesting, but it does not allow you to specify a new tree with a clean user interface. (a) Update the constructor (and other methods if necessary) to accommodate the creation of a tree using a Hash. The constructor should accept a nested structure of Hashes. You should be able to specify a tree as below. To test your functionality, you should traverse (by calling visit_all method on the tree, i.e., the root node) this entire tree and print out the node names. (b)

Write more code which counts the total number of nodes on the tree and also counts the total number of leaves on the tree.

```
ruby_tree = Tree.new({
  'ggp' => {
    'gp1' =>
      { 'p1' => { 'c1' => {} },
        'p2' => { 'c2' => {} , 'c3' => {} }
      },
    'gp2' =>
      { 'p3' => { 'c4' => {} },
        'p4' => { 'c5' => {} , 'c6' => {} }
      },
    'gp3' =>
      { 'p5' => { 'c7' => {} },
        'p6' => { 'c8' => {} , 'c9' => {} }
      }
  }
})
```

Hint: for part (a) you may mainly modify the *initialize* function of the Tree class.

Task #6: In this task, your code creates a random list of 25 shape objects, then traverses the list from start to end, and computes the total area of the shape objects. First you need to implement the class hierarchy diagram of the shape types below. Shape is an abstract class which has only a "color" attribute whereas (regular) Octagon class and (regular) Hexagon class are concrete children of Shape class, and they have more attributes and constructors. Note that you do not know beforehand the order of the shape objects (i.e. Octagons and Hexagons) created in the random list, e.g., you do not know beforehand

whether the 1st item is Octagon or Hexagon. *Writing readme carries 1 point.*

Note. When you traverse the list to calculate the total area, you call the area() function of each shape object (without considering whether it is Octagon or Hexagon). That means, you will use the concept of polymorphism.

Hint: While building the list of shape objects, use rand(2) to generate a random number 0 or 1; if 0, then you may add a Octagon object, else add a Hexagon object to the list.

